

A hybrid optimized deep learning model via the Golden Jackal Optimizer for accurate stock price forecasting

Vasileios Gkonis^{*} , Ioannis Tsakalos

Department of Accounting and Finance, University of Thessaly, Gaiopolis Campus, Larissa GR-41500, Greece

ARTICLE INFO

Keywords:

Golden jackal optimizer (GJO)
Stock prediction
Long short-term memory (LSTM)
Gated recurrent unit (GRU)
Metaheuristic optimization

ABSTRACT

Stock markets have historically been the reference point of the worldwide financial environment throughout the years. Investing in the stock market carries risk; however, it can offer significant short- or long-term returns for the investor. Forecasting stock prices has been a vital topic among professionals and researchers; however, it has always been a difficult task. Machine learning and metaheuristic algorithms over the last years have shown remarkable performance in a variety of sectors, encompassing the financial sector. In this work, we propose an optimized hybrid deep learning model via the Golden Jackal Optimizer (GJO) algorithm for the task of precise stock price prediction. We select the twenty highest stocks in terms of market capitalization from the S&P 500 index as our benchmark dataset. We compare the proposed model with eighteen different neural network models, as well as the PSO-GRU | LSTM and GA-GRU | LSTM models, which refer to optimized deep learning models using the Particle Swarm Optimization (PSO) algorithm and Genetic Algorithms (GA), respectively. The experiments are evaluated based on RMSE, MAE, and MAPE criteria. The results demonstrate that metaheuristic-optimized models outperform standard approaches, with the proposed GJO-GRU | LSTM model enhancing forecasting quality.

1. Introduction

The global economy is significantly influenced by stock markets, where the law of supply and demand influences stock prices (Bosworth et al., 1975). Nevertheless, the technological advancement over the years has increased interest in investing in the stock market via the internet and mobile phones (Fan, 2022). This immediacy and ease have further increased the stock market's demand. Predicting the price of a stock is the ultimate task for investors and firms. Stock prices fluctuate based on several factors, including company performance, economic conditions, and investor behaviour. This behaviour includes risk for potential financial losses when investing in these assets. As anticipated, accurate stock price prediction is essential. Over the past decades, various investors and researchers have studied and evaluated a vast array of candidate methodologies to overcome this challenge. One of the first approaches was the random walk theory (Godfrey et al., 1964; Fama 1995). Moreover, the literature has evaluated the Autoregressive Integrated Moving Average (ARIMA) and Generalized Autoregressive Conditional Heteroskedasticity (GARCH) methods for accurately predicting stocks, including their hybridization and combination (Franses & Van

Dijk, 1996; Chong et al., 1999; Chiang & Doong, 2001; Najand, 2002; Mestel & Gurgul 2003; Awartani & Corradi, 2005; Stevenson, 2007; Jarrett & Kyper, 2011).

However, over the past years, the popularity of machine learning models across a variety of sectors has also inspired scholars to use these types of algorithms in the task of stock price prediction, since these models are able to analyze complex datasets and identify patterns, whereas classical statistical approaches may fail to perform well. Various researchers have evaluated the performance of machine learning approaches for the task of stock price prediction (Guresen et al., 2011; Ballings et al., 2015; Lv et al., 2019; Shahi et al., 2020; Yu & Yan, 2020; Kumbure et al., 2022). Moreover, the existent literature work of several researchers has shown that machine learning models are able to outperform classical statistical methods (Adebiyi et al., 2014; Siami-Namini & Namin, 2018; Sirisha et al., 2022; Almaafi et al., 2023). In addition, Makridakis et al. (2023) provide an extensive comparison between statistical, machine learning, and deep learning models for the task of time series prediction. The authors' findings indicate that the combined deep learning models outperforms standard statistical and machine learning models in terms of performance. Despite their

^{*} Corresponding author.

E-mail addresses: vgkonis@uth.gr (V. Gkonis), i.tsakalos@uth.gr (I. Tsakalos).

<https://doi.org/10.1016/j.eswa.2025.127287>

Received 6 January 2025; Received in revised form 17 February 2025; Accepted 15 March 2025

Available online 20 March 2025

0957-4174/© 2025 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

superiority as predictive methods, the proper selection of these algorithms' hyperparameters significantly influences their performance (Yang & Shami 2020; Morales-Hernández et al., 2023; Bischl et al., 2023).

Metaheuristic algorithms are suitable for tackling complex optimization problems, as they can generate near-optimal or optimal solutions within a shorter timeframe, and are generally categorized in two different methods. In single-solution-based methods, an initial solution—often generated randomly—is continuously updated until a stopping criterion is met. Population-based methods start with a set of randomly generated solutions, which are continuously updated by sharing information among population, aiming to identify a near-optimal or optimal solution. (Zervoudakis & Tsafarakis, 2020; Zervoudakis & Tsafarakis, 2023). Metaheuristic algorithms mimic various phenomena in the nature such as evolutionary processes, swarm intelligence, laws of physics, and even human behaviour (Abdollahzadeh et al., 2024). Four main points account for the increasing development of these types of algorithms (Mirjalili et al., 2014). Firstly, these algorithms are built on simple concepts, allowing scientists to propose new or even modify existing one. Moreover, metaheuristics can be applied to different problems without modifying their structure. The researcher only needs to define the problem in terms of inputs and evaluate the outputs. Additionally, most metaheuristic algorithms have derivation-free mechanisms, which can be useful for problems where derivatives are unavailable, or their computation is too complex. Finally, their stochastic nature helps prevent stagnation in local optima, which is crucial for tackling real-world problems that are often highly complex. The number of available metaheuristic algorithms is enormous; in the past years, over five hundred algorithms have been proposed (Rajwar et al., 2023). Metaheuristic algorithms have been widely utilized to hyperparameter optimization in machine learning (Abd Elaziz et al., 2021; Akay et al., 2022; Morales-Hernández et al., 2023; Narayanan & Ganesh, 2024). Except for the concept of stock price prediction, the use of machine learning methods and metaheuristic algorithms has been widely applied in a variety of financial tasks. (Marqués et al., 2013; Ertenlice & Kalayci, 2018; Andriosopoulos et al., 2019, Liu & Wang, 2024).

In this work we propose the GJO-GRU | LSTM model, which refers to an optimized deep learning model via Golden Jackal Optimization algorithm (Chopra & Ansari, 2022), a swarm-based metaheuristic optimization algorithm that takes its inspiration from the golden jackals' collaborative hunting behaviour in the nature. In the majority works of the existent literature, the main task is to optimize a specific architecture of neural network, which usually comprises of one or more layers of Gated Recurrent Unit (GRU) or Long Short-Term Memory (LSTM) networks. As far as we know, there is limited application, that try to optimize the network architecture, where both GRU or LSTM (GRU | LSTM) can be selected. In addition, to the best of our knowledge, other research tends to optimize only the components of model, and not the batch size, which can play a vital role in model's performance. The main purpose of our approach is to find a way, to create optimal tailored models, for each stock. The results show that the suggest model provides high prediction accuracy. The application, of GJO is comparatively limited in the literature, in the comparison to other metaheuristic optimization algorithms. This work is probably the first one which evaluates its performance regarding hyperparameter optimization, for the task of stock price prediction. This research contributes in several ways, providing valuable assistance to both researchers and investors. At first, we propose a robust model for efficient stock price prediction. Moreover, we further enrich the literature in the field of hyperparameter optimization in machine learning, as well as the broader topic of metaheuristic optimization.

The remainder of this work is organized as following. Section 2 provides the extant literature on deep learning and metaheuristic optimized machine learning models. In Section 3, we present the methods that will be evaluated in this work. In Section 4, we present the evaluation metrics, experimental setup as well as the corresponding results. Finally, in

Section 5 we conclude and discuss future directions.

2. Literature review

The existing literature includes numerous works that seek to identify models for accurate stock price forecasting. In this section, at first, we discuss the applications of classical LSTM and GRU networks. Next, we provide some of the extant literature that discusses the concept of metaheuristic optimized models.

2.1. Applications of LSTM and GRU networks in stock price forecasting

Several machine learning models have been applied to the task of accurate stock price forecasting. Over the past years LSTM and GRU have shown, that are able to surpass the classical machine models for the concept of stock price forecasting. Fischer and Krauss (2018) proposed the use of LSTM networks for predicting directional movements for the stocks of the S&P500. LSTM outperformed other well-known models, such as Random Forests (RF), Deep Neural Networks (DNN), and Logistic Regression (LR). LSTM networks exhibit low exposure to systematic risk compared to benchmark models, highlighting their potential in financial time series prediction. Nabipour et al. (2020) evaluate the performance of several tree-based models as well as Artificial Neural Networks (ANN) as well as Recurrent Neural Network (RNN) and LSTM, for the concept of stock Market Prediction utilizing a ten-year dataset of stocks which consisted of four different market groups. Once again, LSTM surpassed the rest models.

Di Persio & Honchar (2017) compared multiple RNN architectures, including multi-layer RNNs, LSTM, and GRU in forecasting Google stock price movements over various time horizons. The findings indicate that LSTM can effectively handle long sequences and achieve good performance. Forecasting the price of cryptocurrencies is a challenging task due to the large volatility of the respective market. Gao et al. (2021) proposed a stock market prediction model integrating technical indicators, investor sentiment, and financial data. The authors applied LASSO and PCA methods, for dimension reduction and evaluate LSTM and GRU models for forecasting. Their results show that LSTM and GRU perform similarly in predicting stock prices. Additionally, models using LASSO outperform those using PCA in prediction accuracy. Dutta et al., (2020) compare the performance of RNN, LSTM, and GRU models for Bitcoin price prediction, incorporating exogenous and endogenous factors. Their results show that GRU with recurrent dropout outperforms other models based on RMSE evaluation. Additionally, the results demonstrated that simple trading strategies combined with GRU model can lead to financial gain.

Hamayel and Owda (2021) explored the use of recurrent neural network RNN models for predicting cryptocurrency prices. The authors compared the performance of GRU, LSTM, and bidirectional LSTM (bi-LSTM) models. The results showed that the GRU model achieved the best performance, whereas the bi-LSTM model showed the worst prediction accuracy. Various researchers incorporate sentiment analysis with LSTM networks, where this combination has shown promising performance. Fang et al., (2024) proposed a VIX-Lasso-GRU Model for intraday stock index futures forecasting in China. The model enhances a GRU network by incorporating VIX information to reduce prediction errors and Lasso for faster training. They apply it to 5-minute closing price predictions of three index futures datasets. Results show that VIX-Lasso-GRU outperforms pure GRU and LSTM in both speed and accuracy. This study highlights the benefits of volatility indices and feature selection in high-frequency trading. Liu et al., (2021) combines LSTM model with data from online social networks. More specifically, the authors extracted data from *EastyMoney* and generated daily social networks. Except the use of traditional variables, as input for LSTM networks the authors also used the network variable for each stock. The results demonstrated that the proposed approach enhanced forecasting quality for predicting the close prices of SSE 50 constituent stocks.

Table 1

The ten stocks and their respective tickers evaluated in this work.

Stock	Ticker
Apple Inc.	AAPL
Amazon.com Inc	AMZN
Broadcom Inc.	AVGO
Berkshire Hathaway Class B	BRK.B
Costco Wholesale Corp	COST
Alphabet Inc. Class A	GOOGL
Home Depot, Inc.	HD
JPMorgan Chase & Co.	JPM
Eli Lilly & Co.	LLY
Mastercard Incorporated	MA
Meta Platforms, Inc. Class A	META
Microsoft Corp	MSFT
Netflix Inc	NFLX
Nvidia Corp	NVDA
Procter & Gamble Company	PG
Tesla, Inc.	TSLA
United health Group Incorporated	UNH
Visa Inc.	V
Walmart Inc.	WMT
Exxon Mobil Corporation	XOM

Table 2

Descriptive statistics of evaluated dataset.

Asset	Mean	Std	Min	Max
AAPL	134.902	42.393	46.594	216.419
AMZN	136.244	31.346	81.820	197.850
AVGO	55.863	31.555	14.612	181.710
BRK-B	284.296	64.341	162.130	420.520
COST	442.842	141.243	240.886	867.400
GOOGL	107.671	31.101	52.576	185.164
HD	275.880	51.370	135.782	388.059
JPM	128.569	28.783	69.256	203.661
LLY	308.664	196.764	99.127	907.772
MA	345.928	53.699	197.666	485.879
META	264.153	94.670	88.643	526.315
MSFT	260.098	79.734	125.861	452.035
NFLX	425.647	127.973	166.370	691.690
NVDA	25.904	24.259	3.698	135.568
PG	130.253	17.114	86.343	165.563
TSLA	187.361	95.931	14.093	409.970
UNH	400.813	102.971	181.838	542.595
V	211.912	29.689	130.999	288.220
WMT	45.455	6.937	32.178	68.552
XOM	70.379	27.729	24.809	118.210

Table 3

Optimization space.

Architecture	Optimization Range
GRU LSTM	[1–50]
Dropout	[0.2–0.7]
GRU LSTM (optional layer)	[1–50]
Dropout (optional layer)	[0.2–0.7]
Batch Size	[4,50]
Optimizer	[RMSPProp, Adam, RAdam]

Liu et al. (2024) proposed the LSTM-GRU-SA-AM model for predicting stock prices using of new energy vehicle (NEV) companies integrating multi-source data. The model integrates LSTM, GRU, Self-Attention (SA), and an Attention Mechanism (AM) to enhance forecasting accuracy. They validate their approach on four Chinese NEV stocks and compare it with six baseline models. Results show that multi-source data significantly improves prediction accuracy, and their proposed model outperforms all others. Wu et al. (2022) proposed the S_I_LSTM, which combines several data sources and sentiment analysis of investors to improve prediction accuracy. At first, the authors extract

Table 4

Description of Benchmarks Models.

Models	Architecture	Parameters
LSTM	LSTM	10
	Dropout	0.2
	Optimizer	[RMSPProp, Adam, RAdam]
GRU	Batch Size	16
	GRU	10
	Dropout	0.2
LSTM2D	Optimizer	[RMSPProp, Adam, RAdam]
	Batch Size	16
	LSTM	10
GRU2D	Dropout	0.2
	GRU	10
	Dropout	0.2
LSTM-GRU	Optimizer	[RMSPProp, Adam, RAdam]
	Batch Size	16
	LSTM	10
GRU-LSTM	Dropout	0.2
	GRU	10
	Dropout	0.2
	Optimizer	[RMSPProp, Adam, RAdam]
	Batch Size	16
	LSTM	10
	Dropout	0.2
	Optimizer	[RMSPProp, Adam, RAdam]
	Batch Size	16

and process information such as stock data, technical indicators as well as non-traditional data like posts and news. Then a Convolutional Neural Network (CNN) is employed to create the investor index from the non-traditional data. Finally, they used the respective index along with historical data and technical indicators in order to predict stock prices via LSTM. The proposed model, evaluated in the China Shanghai A-share market, and the results demonstrated that S_I_LSTM is able to provide better results in contrast to traditional methods. Jiang (2021) provides an overview of applications in deep learning models for stock market prediction. The author categorizes various neural network architectures, data sources, and evaluation metrics, while also addressing implementation and reproducibility challenges.

2.2. Metaheuristic optimized models

Ji et al., (2021) proposed the IPSO-LSTM, which corresponds to an optimized LSTM model via the use of an Improved Particle Swarm Optimization (IPSO) algorithm. The proposed IPSO includes an adaptive mutation factor to avoid premature convergence as well as the authors presented a non-linear approach to improve the inertia weight. The proposed method, evaluated on the Australian stock market index. The results showed that IPSO-LSTM, provided better accuracy than Support Vector Regression (SVR), LSTM, and PSO-LSTM.

Chung & Shin (2018) proposed a hybrid approach that combines LSTM networks with Genetic Algorithms (GA)(Holland,1992) to improve prediction accuracy. The model is evaluated using daily data from the Korea Stock Price Index (KOSPI), and results indicate that the hybrid GA-LSTM model significantly outperforms benchmark models.

Mu et al., (2023) proposed the MS-SSA-LSTM model, which combines multi-source data, sentiment analysis, the Sparrow Search Algorithm (SSA), and LSTM network. Using data from *EastMoney* forum, the algorithm initially generates a sentiment index. The SSA is then the hyperparameter optimization technique utilized for optimizing the hyperparameters of LSTM. The suggested approach outperformed

Table 5

Performance results of GJO-GRU | LSTM, PSO-GRU|LSTM and GA-GRU|LSTM per asset.

Asset	GJO-GRU LSTM			PSO-GRU LSTM			GA-GRU LSTM		
	RMSE	MAE	MAPE	RMSE	MAE	MAPE	RMSE	MAE	MAPE
AAPL	2.652	1.943	1.059	2.695	1.973	1.074	3.843	2.901	1.584
AMZN	3.04	2.316	1.487	3.242	2.489	1.586	3.133	2.401	1.531
AVGO	3.75	2.526	2.133	3.895	2.604	2.197	4.452	2.942	2.480
BRK-B	3.432	2.728	0.72	3.948	3.099	0.814	3.685	2.934	0.774
COST	14.301	11.628	1.690	16.636	13.444	1.959	16.376	13.464	1.987
GOOGL	3.292	2.546	1.741	2.658	1.902	1.321	3.394	2.623	1.811
HD	4.699	3.640	1.117	4.747	3.735	1.145	4.721	3.663	1.124
JPM	3.088	2.289	1.376	3.298	2.619	1.486	2.305	1.745	1.022
LLY	23.258	19.093	2.789	21.162	16.273	2.395	14.866	11.634	1.841
MA	6.713	5.568	1.277	11.045	9.260	2.125	6.765	5.497	1.288
META	9.463	6.208	1.605	10.894	7.595	1.949	10.712	7.440	1.895
MSFT	5.12	4.069	1.078	6.810	5.698	1.484	5.965	4.888	1.291
NFLX	12.426	8.718	1.749	11.683	8.155	1.606	12.100	8.443	1.704
NVDA	4.403	2.838	3.483	3.509	2.314	3.202	3.207	2.241	3.278
PG	1.309	0.984	0.654	1.467	1.100	0.734	1.457	1.131	0.749
TSLA	6.979	5.065	2.397	6.990	5.140	2.445	7.543	5.645	2.678
UNH	8.502	6.166	1.256	7.709	5.461	1.112	8.734	6.465	1.308
V	2.867	2.360	0.911	2.314	1.866	0.727	4.841	4.134	1.586
WMT	0.798	0.56	0.99	0.867	0.635	1.099	0.852	0.624	1.086
XOM	1.484	1.168	1.120	1.925	1.516	1.449	1.690	1.339	1.277

Note: The MAPE values are expressed as percentages.

conventional multi-layer perceptron (MLP), CNN, LSTM, MS-LSTM, and SSA-LSTM when the authors compared their performance in all equities traded on China's A-share market. [Yu et al. \(2023\)](#) points out the importance of the realized volatility (RV) of the stock price index. To that end, the authors proposed the GVMD-Q-DBN-LSTM-GRU method. The constructed hybrid model combined an optimized Variation Model Decomposition (VMD) via the use of Grey Wolf Optimizer (GWO) algorithm to acquire the Intrinsic Mode Functions (IMFs). Next, the same IMF is predicted concurrently with Deep Belief Network (DBN), LSTM, and GRU. Finally, the Q-Learning algorithm is applied in order to define the optimal weights of the aforementioned three models, in order to build an integrated model. The authors used the RV of three different indices. The proposed model is able to provide better performance, in contrast to other neural network models.

[Shahvaroughi Farahani & Razavi Hajiagha \(2021\)](#) proposed an approach of metaheuristic optimized ANN with the use of Social Spider Algorithm (SSA) and Bat algorithm (BA). Moreover, the authors employed twenty technical indicators as inputs and then applied GA for feature selection. Furthermore, they compared the performance of metaheuristic optimized ANN with traditional ARMA and ARIMA models, utilizing dataset of the five most important and international indices. The results showed that metaheuristic optimized ANN provide better forecasting quality. [Xiaopeng \(2024\)](#) proposed the AO-GRU model, for the concept of accurate stock price forecasting. The respective model implies a combined GRU with Aquila optimizer (AO). The author compares its performance with standard GRU, ALO-GRU which refers to an optimized GRU, using the Ant Lion Optimizer (ALO) and GWO-GRU. Based on the experiments utilized in the Hang Seng index price data, the AO-GRU achieved better performance.

[Gülmez \(2023\)](#) proposed an optimized LSTM model with Artificial Rabbit Optimizer (ARO) for stock prediction of the Dow Jones Industrial Average (DJIA) index stocks. The proposed approach outperformed ANN, standard LSTM models, and an optimized LSTM via GA (GA-LSTM). [Ribeiro et al. \(2021\)](#) proposed the HAR-PSO-ESN model which combines the feature design of heterogeneous autoregressive (HAR) model, the predictive strength of echo state network (ESN) and the employment of PSO ([Kennedy & Eberhart, 1995](#)) for the hyper-parameter optimization task. The authors compared the proposed approach with several benchmark models in predicting daily realized volatilities of three Nasdaq stocks, and the results demonstrated that HAR-PSO-ESN provides more accurate predictions in most scenarios. [Kumar et al., \(2022a\)](#) proposed an optimized LSTM with and adaptive

PSO hybrid model for predicting the stock prices of three major stock indices. The authors compared the proposed model with GA-LSTM, Elman Neural Network (ENN) model and standard LSTM, and the outcomes shows, the superiority of the model. [Chou et al. \(2022\)](#) proposed an intelligent candlestick forecast system, via a metaheuristic optimized multi-output least squares support vector regression (MLSSVR) model. For optimization purpose the Forensic-Based Investigation (FBI), Symbiotic Organisms Search (SOS) and the Teaching-Learning-Based Optimization (TLBO) were employed. The results show, that FBI-MLSSVR outperforms the rest models in terms of forecasting accuracy.

[Kumar et al. \(2022b\)](#) propose a hybrid feed-forward neural network combining discrete particle swarm optimization (DPSO), particle swarm optimization (PSO), and the Levenberg–Marquardt (LM) algorithm for stock price forecasting. DPSO selects optimal features and hidden layer neurons, while PSO fine-tunes weights and biases. The model is tested on major indices and compared with a genetic algorithm-based approach, where it showed its promising performance. [Göçken et al. \(2016\)](#) evaluate the effectiveness of technical indicators in forecasting the Turkish stock market using hybrid Artificial Neural Network (ANN) models. The approach integrates Harmony Search (HS) and GA to select relevant indicators and optimize the number of hidden neurons, addressing overfitting and underfitting issues. The models are compared using loss functions, investment returns, and graphical analysis, with results showing that the HS-based ANN model outperforms others in stock market forecasting.

3. Materials and methods

This section presents the components of GJO-GRU | LSTM. At first, we introduce the concept of LSTM and GRU neural networks following the mathematical notation of [Olah \(2015\)](#). In the last subsection, we extensively present the GJO, as it is the proposed metaheuristic algorithm in this work.

3.1. LSTM

RNN is a well-known deep learning architecture, which in contrast to Feed-Forward Neural Networks (FFNN), RNN incorporate feedback loops to process sequential data, utilizing a hidden state to retain previous information. However, RNN have difficulties when dealing with long-term dependencies. This type of neural networks, suffer from the “*vanishing gradient problem*” where during backpropagation through

Table 6

RMSE results for benchmark models.

Asset	GRU	GRU2D	GRU-LSTM	LSTM	LSTM2D	LSTM-GRU
RMSProp						
AAPL	5.572	13.867	7.910	4.515	10.978	7.420
AMZN	3.831	5.553	9.469	3.907	8.221	4.119
AVGO	21.286	23.884	17.229	18.970	33.672	14.409
BRK-B	9.678	17.896	15.271	11.736	15.164	12.201
COST	33.421	105.735	73.251	56.833	71.483	43.622
GOOGL	5.708	9.570	6.943	4.495	8.790	6.397
HD	6.677	12.133	19.767	6.207	10.037	8.955
JPM	11.500	6.529	8.649	3.269	7.564	4.575
LLY	108.183	158.704	195.283	90.868	180.377	207.502
MA	7.062	16.430	28.963	8.223	24.218	16.128
META	19.092	22.521	33.065	42.206	35.811	16.477
MSFT	22.388	31.689	30.206	31.429	27.849	20.720
NFLX	19.596	21.044	20.728	19.098	22.499	24.626
NVDA	25.553	25.664	21.264	22.098	25.070	24.718
PG	2.215	2.480	4.577	2.862	2.646	2.223
TSLA	10.120	11.946	10.627	9.791	15.157	10.332
UNH	13.256	18.703	17.509	11.883	17.722	14.282
V	9.981	9.795	10.571	7.824	4.563	4.496
WMT	2.064	6.470	3.909	1.726	2.720	2.523
XOM	3.162	3.479	2.425	2.843	4.565	2.478
Adam						
AAPL	6.452	8.187	9.778	5.692	8.474	9.716
AMZN	4.237	6.474	4.949	4.477	7.235	7.105
AVGO	11.084	29.747	15.007	25.583	29.358	17.516
BRK-B	13.059	12.766	24.074	17.143	17.057	18.042
COST	110.765	65.293	83.506	86.063	51.475	118.197
GOOGL	7.277	11.088	7.167	5.188	7.309	7.311
HD	18.707	11.224	8.988	11.177	12.037	11.215
JPM	9.477	9.893	6.553	11.764	8.282	7.124
LLY	198.037	252.875	124.186	154.298	237.301	158.045
MA	17.269	17.338	23.756	20.818	22.648	14.388
META	14.037	35.356	29.507	28.389	31.250	40.377
NFLX	26.844	52.032	20.271	19.865	21.593	24.323
MSFT	23.537	31.871	35.690	34.788	25.240	21.377
NVDA	22.087	29.675	24.588	25.752	32.330	25.090
PG	1.986	3.821	2.977	2.857	3.334	3.449
TSLA	8.819	15.204	8.939	10.104	11.987	13.317
UNH	20.985	15.616	15.235	13.027	14.322	16.145
V	6.384	7.434	9.702	6.452	4.913	12.938
WMT	6.420	3.258	4.547	2.821	5.661	4.552
XOM	2.968	3.331	2.422	2.942	3.294	4.399
Radam						
AAPL	6.841	10.475	14.193	6.334	10.495	7.240
AMZN	5.369	8.210	3.995	6.080	9.272	7.932
AVGO	36.879	24.938	22.522	26.701	25.065	19.446
BRK-B	14.278	19.013	21.406	12.453	17.984	8.599
COST	143.899	141.369	36.577	45.555	118.484	36.931
GOOGL	14.200	9.929	8.040	4.744	8.123	6.917
HD	10.337	8.854	10.264	12.717	22.446	9.270
JPM	5.627	25.595	6.434	2.835	9.960	5.063
LLY	89.022	205.455	125.044	172.188	181.769	118.145
MA	20.128	27.834	10.161	16.058	15.367	15.401
META	41.294	19.160	18.287	14.303	61.023	17.141
MSFT	13.509	30.309	22.840	18.289	25.213	14.970
NFLX	22.730	17.525	20.720	24.584	21.036	17.675
NVDA	27.354	30.770	16.169	14.090	25.591	25.336
PG	4.508	2.406	1.759	3.135	2.671	4.858
TSLA	11.806	11.523	10.186	10.018	12.236	10.163
UNH	15.959	14.512	15.546	14.342	17.252	10.971
V	6.971	8.394	12.633	6.871	12.622	5.711
WMT	4.240	2.739	1.983	2.013	4.159	1.008
XOM	3.618	3.913	2.891	2.790	3.816	2.475

Table 7

MAE results for benchmark models.

Asset	GRU	GRU2D	GRU-LSTM	LSTM	LSTM2D	LSTM-GRU
RMSProp						
AAPL	4.268	11.262	6.194	3.432	8.656	5.817
AMZN	3.011	4.509	8.495	3.049	6.991	3.176
AVGO	16.736	18.336	12.793	14.132	27.988	10.445
BRK-B	8.154	15.044	12.683	9.620	12.956	10.085
COST	26.313	85.116	56.452	42.975	55.952	33.969
GOOGL	4.567	7.492	5.544	3.539	7.020	5.178
HD	5.346	10.206	16.893	4.962	8.207	7.304
JPM	9.213	5.252	6.808	2.614	6.017	3.837
LLY	89.225	135.241	172.891	78.562	156.865	183.127
MA	5.727	13.979	24.790	6.632	20.573	13.702
META	14.857	18.127	25.052	31.177	27.704	11.687
MSFT	18.510	27.190	25.350	25.808	22.995	16.831
NFLX	15.392	16.456	16.429	15.080	17.553	20.377
NVDA	18.386	17.112	14.396	14.410	17.642	16.034
PG	1.702	2.029	3.803	2.356	2.187	1.755
TSLA	7.663	9.346	8.339	7.567	11.901	7.905
UNH	10.567	15.664	14.837	9.133	14.996	11.702
V	8.651	8.420	9.182	6.747	3.654	3.557
WMT	1.636	5.132	3.030	1.237	1.946	1.912
XOM	2.536	2.789	1.966	2.340	3.589	2.007
Adam						
AAPL	5.097	6.371	7.624	4.469	6.585	7.534
AMZN	3.258	5.314	4.024	3.592	6.055	5.914
AVGO	7.552	23.890	10.939	20.814	24.772	12.699
BRK-B	11.060	10.912	20.477	14.179	13.915	14.945
COST	84.529	52.181	65.189	65.348	41.311	92.958
GOOGL	5.694	8.166	5.505	4.207	5.894	5.916
HD	15.728	9.327	7.404	9.288	10.216	9.113
JPM	7.538	8.084	5.463	9.146	6.879	6.025
LLY	174.137	228.782	104.664	128.827	211.097	134.403
MA	14.729	15.096	19.924	17.863	19.650	12.105
META	9.887	26.963	22.813	21.977	23.950	31.276
NFLX	21.897	42.421	15.651	15.447	17.230	18.948
MSFT	20.025	26.738	30.080	28.830	20.998	17.647
NVDA	14.576	21.600	16.357	17.196	23.382	17.273
PG	1.560	3.189	2.479	2.393	2.807	2.887
TSLA	6.664	12.004	6.789	7.773	9.393	10.598
UNH	17.771	12.898	12.650	10.329	11.630	13.537
V	5.484	6.535	8.517	5.609	4.037	11.286
WMT	4.855	2.428	3.595	2.260	4.488	3.649
XOM	2.370	2.647	1.974	2.438	2.658	3.581
Radam						
AAPL	5.469	8.156	12.400	4.950	8.182	5.684
AMZN	4.400	6.881	3.128	5.036	7.861	6.706
AVGO	30.146	19.738	17.875	19.737	19.812	14.902
BRK-B	11.826	15.925	18.709	10.466	15.301	6.958
COST	117.443	116.833	27.894	35.462	96.035	26.781
GOOGL	12.318	7.572	6.525	3.682	6.506	5.571
HD	8.305	7.202	8.558	10.628	19.048	7.734
JPM	4.801	21.419	5.230	2.134	7.928	4.222
LLY	73.237	178.104	104.789	147.484	157.971	96.406
MA	17.124	24.618	8.579	13.456	13.355	13.237
META	31.719	14.355	14.046	10.536	46.291	12.859
MSFT	10.909	25.377	19.023	15.111	21.154	12.192
NFLX	18.344	13.146	16.118	20.318	15.913	13.401
NVDA	18.928	22.457	10.511	8.472	17.609	17.390
PG	3.734	1.936	1.366	2.573	2.204	4.052
TSLA	9.269	9.085	7.776	7.660	9.673	7.664
UNH	13.078	12.054	13.043	11.752	13.497	8.553
V	5.891	7.212	11.059	6.103	11.117	4.845
WMT	3.160	2.082	1.392	1.519	3.049	0.718
XOM	2.887	3.132	2.347	2.228	3.042	2.037

time, the gradients may have explosive or vanishing behavior which poses a challenge for the network to effectively capture long-term dependencies (Hochreiter, 1991). Hochreiter & Schmidhuber (1997) introduced the LSTM networks, where tackles the drawbacks of classical RNN networks.

The sophisticated architecture of LSTM contains memory blocks, which are connected between layers, and each block consists of gates that control the flow of information. Within a unit, there are cell states

and three types of gates: forget, input, and output gates. For each gate, the calculation of the respective output involves the sigmoid transformation of the input of the current timestep x_t , the output of the previous timestep (h_{t-1}), along with the respective weight matrices and bias vectors.

The forget gate (f_t) (Gers et al.,2000) (Eq.1) determines which information should be discarded from the cell state (c_t) (Eq. (2) of the previous timestep (c_{t-1}). Next the procedure continues, with the update

Table 8

MAPE results for benchmark models.

Asset	GRU	GRU2D	GRU-LSTM	LSTM	LSTM2D	LSTM-GRU
RMSProp						
AAPL	2.310	5.930	3.271	1.873	4.549	3.081
AMZN	1.898	2.756	5.326	1.931	4.265	2.042
AVGO	13.486	14.543	10.044	11.069	22.964	8.225
BRK-B	2.110	3.844	3.232	2.480	3.325	2.587
COST	3.692	11.907	7.798	5.918	7.771	4.751
GOOGL	3.040	4.900	3.656	2.394	4.618	3.457
HD	1.638	3.112	5.011	1.523	2.516	2.236
JPM	5.103	2.962	3.768	1.512	3.350	2.221
LLY	12.497	19.112	24.830	11.225	22.361	26.269
MA	1.344	3.181	5.598	1.547	4.653	3.118
META	3.574	4.596	5.726	6.933	6.347	3.005
MSFT	4.641	6.851	6.353	6.438	5.773	4.261
NFLX	2.955	3.176	3.223	3.067	3.671	3.976
NVDA	22.346	19.599	16.724	16.282	21.084	18.021
PG	1.139	1.325	2.459	1.553	1.435	1.168
TSLA	3.594	4.399	3.881	3.549	5.659	3.728
UNH	2.147	3.133	2.955	1.856	3.010	2.357
V	3.276	3.185	3.465	2.557	1.400	1.396
WMT	2.779	8.666	5.100	2.094	3.267	3.221
XOM	2.368	2.608	1.894	2.190	3.319	1.925
Adam						
AAPL	2.714	3.368	4.004	2.385	3.478	3.963
AMZN	2.099	3.313	2.511	2.268	3.716	3.645
AVGO	5.949	19.295	8.605	16.947	20.525	9.920
BRK-B	2.847	2.815	5.224	3.612	3.543	3.806
COST	11.623	7.283	9.034	8.976	5.796	12.893
GOOGL	3.760	5.261	3.605	2.828	3.901	3.938
HD	4.682	2.859	2.261	2.800	3.081	2.807
JPM	4.214	4.534	3.094	5.048	3.902	3.429
LLY	24.952	33.213	14.746	18.050	30.385	18.988
MA	3.347	3.453	4.496	4.050	4.470	2.782
META	2.633	6.167	5.236	5.056	5.502	7.139
NFLX	4.298	7.656	3.264	3.025	3.474	3.986
MSFT	5.046	6.708	7.532	7.193	5.285	4.456
NVDA	16.681	26.485	18.742	19.743	28.432	20.350
PG	1.040	2.079	1.622	1.568	1.834	1.891
TSLA	3.142	5.652	3.203	3.722	4.390	4.938
UNH	3.544	2.618	2.552	2.103	2.360	2.730
V	2.135	2.500	3.221	2.133	1.570	4.260
WMT	8.132	4.073	6.071	3.835	7.582	6.179
XOM	2.244	2.498	1.904	2.285	2.494	3.327
Radam						
AAPL	2.918	4.281	6.593	2.623	4.299	3.013
AMZN	2.712	4.185	1.987	3.114	4.750	4.090
AVGO	24.542	15.873	14.408	15.401	15.916	11.891
BRK-B	3.018	4.062	4.801	2.689	3.919	1.816
COST	16.495	16.482	3.897	4.955	13.462	3.697
GOOGL	8.183	4.916	4.292	2.521	4.287	3.706
HD	2.573	2.214	2.573	3.179	5.628	2.337
JPM	2.758	11.954	2.942	1.292	4.396	2.417
LLY	10.253	25.331	14.712	20.883	22.500	13.420
MA	3.882	5.593	1.973	3.053	3.062	3.015
META	7.179	3.752	3.425	2.721	10.315	3.219
MSFT	2.785	6.368	4.776	3.810	5.324	3.134
NFLX	3.681	2.727	3.372	4.006	3.319	2.741
NVDA	22.393	27.512	12.000	9.484	20.646	20.357
PG	2.424	1.281	0.908	1.704	1.445	2.615
TSLA	4.305	4.220	3.651	3.657	4.474	3.606
UNH	2.675	2.417	2.622	2.378	2.784	1.732
V	2.253	2.720	4.169	2.336	4.200	1.853
WMT	5.283	3.501	2.363	2.569	5.085	1.237
XOM	2.709	2.906	2.196	2.119	2.838	1.949

Note: The MAPE values are expressed as percentages.

of the current c_t . For that purpose, candidate values $\tilde{c}_t$ are calculated following Eq. (3). However, the quantity of the new information that will be stored in the c_t is defined from the element-wise multiplication of $\tilde{c}_t$ with the output of input gate (i_t) (Eq. 4). Following the aforementioned, as it can be also observed in Eq. (2) the current cell state consists of information from the previous timestep and the current new information. Finally the determination of the current hidden state (h_t) (Eq. (5)) is the result of the element-wise multiplication between the output

gate o_t (Eq. (6)) and the transformed c_t through the hyperbolic tangent activation function. The h_t not only serves as the output for the current time step but also as the input for the next time step (h_{t-1}), allowing the network to carry forward relevant information through the sequence. As with the traditional FFNN the weight matrices and bias vectors are updated during the training procedure in order to minimize the respective loss function.

$$f_t = \sigma(W_f \bullet [h_{t-1}, x_t] + b_f) \quad (1)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (2)$$

$$\tilde{c}_t = \tanh(W_c \bullet [h_{t-1}, x_t] + b_c) \quad (3)$$

$$i_t = \sigma(W_i \bullet [h_{t-1}, x_t] + b_i) \quad (4)$$

$$h_t = o_t \odot \tanh(c_t) \quad (5)$$

$$o_t = \sigma(W_o \bullet [h_{t-1}, x_t] + b_o) \quad (6)$$

Where:

W_f, W_i, W_o, W_c : Weight matrices for each gate and the cell state.
 b_f, b_i, b_o, b_c : Corresponding bias vectors.

3.2. GRU

The revolutionary approach in the field of RNNs through LSTM networks has inspired numerous researchers. [Cho et al., \(2014\)](#) introduced GRU networks, which are a modification of LSTM. In contrast to LSTM, GRU include two gates and have fewer parameters. Unlike LSTM, GRU do not incorporate cell states. The update gate (z_t) (Eq. (7)) determines how much information from the previous timestep (h_{t-1}) should be combined with the new information (x_t). The reset gate (r_t) (Eq. (8)) decides how much of the previous hidden state (h_{t-1}) should be forgotten. Next, the candidate hidden state/current memory content ($\tilde{h}_t$) (Eq. 9) is computed by combining the reset gate with x_t and h_{t-1} . Finally, the current hidden state (h_t) is being calculated from both h_{t-1} and $\tilde{h}_t$, with the update gate controlling this combination via element-wise multiplication, as shown in Eq. (10).

$$z_t = \sigma(W_z \bullet [h_{t-1}, x_t]) \quad (7)$$

$$r_t = \sigma(W_r \bullet [h_{t-1}, x_t]) \quad (8)$$

$$\tilde{h}_t = \tanh(W \bullet [r_t \odot h_{t-1}, x_t]) \quad (9)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (10)$$

3.3. Golden jackal Optimizer

The golden jackal is a medium-sized terrestrial predator. The jackal's small size and elongated legs enable it to traverse extensive distances while looking for prey. They consume both animal and plant matter, including rodents, juvenile gazelles, terrestrial birds, hares, reptiles, amphibians, fruit, fish, and insects. Golden jackals exhibit monogamous behavior and reside in copulating pairs. Their families comprise one or two "helpers" who reside with the parents for around one year and care for the succeeding child. They offer security and sustenance to the pups, while assistants utilize predator barks and rumbling growls to alert them to seek refuge. They provide sustenance to a nursing mother, enhancing child-rearing by enabling parents to hunt individually or in pairs. Golden jackals employ a diverse array of howls to communicate, facilitating both hunting and social bonding. Foraging families constantly occupy territories spanning several square kilometers, with sections marked by urine to deter intruders. Cooperative foraging is crucial for jackals to pursue larger prey within accessible ranges. The main phases

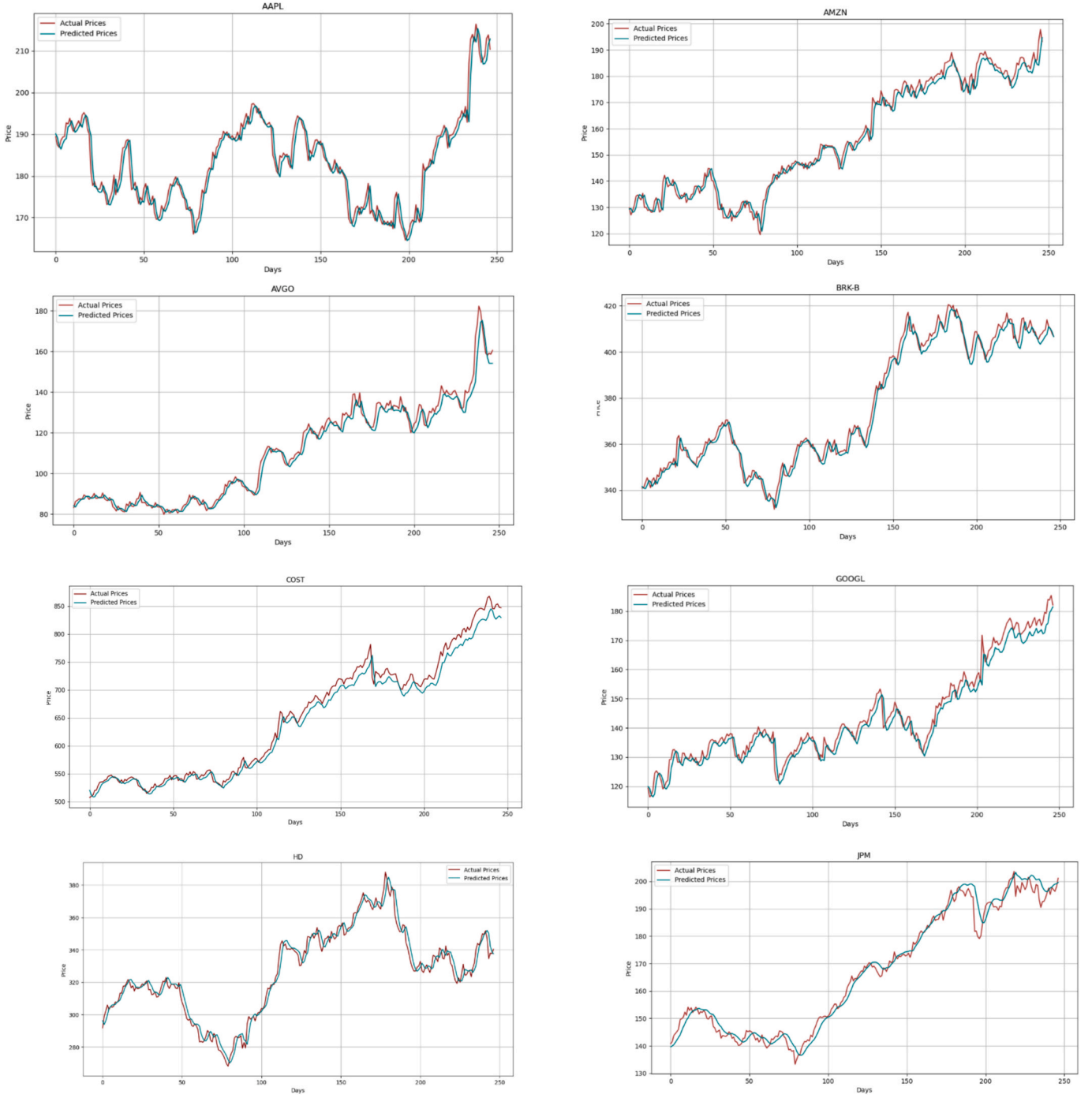


Fig. 1. GJO-GRU/LSTM Results.

of golden jackal's pair hunting include, searching and moving towards the prey, enclosing and aggravating the prey until it stops moving, then pouncing towards it.

3.3.1. Formulation of the search domain

As the first trial, the initial solution is uniformly distributed throughout the search space (Eq. (11)).

$$Y_0 = Y_{min} + rand(Y_{max} - Y_{min}) \quad (11)$$

Where *rand* represents a random vector of uniform distribution, and Y_{min} and Y_{max} corresponds to lower and upper bound respectively. The

initialization produces the *Prey* matrix (Eq. (12)) where the first and second best solution is the male and female jackal respectively. The F_{OA} matrix (Eq. (13)), estimates the fitness value per prey

$$Prey = \begin{bmatrix} Y_{1,1} & Y_{1,2} & \cdots & Y_{1,d} \\ Y_{2,1} & Y_{2,2} & \cdots & Y_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ Y_{n,1} & Y_{n,2} & \cdots & Y_{n,d} \end{bmatrix} \quad (12)$$

$$F_{OA} = \begin{bmatrix} f(Y_{1,1}; Y_{1,2}; \cdots; Y_{1,d}) \\ f(Y_{2,1}; Y_{2,2}; \cdots; Y_{2,d}) \\ \vdots \\ f(Y_{n,1}; Y_{n,2}; \cdots; Y_{n,d}) \end{bmatrix} \quad (13)$$

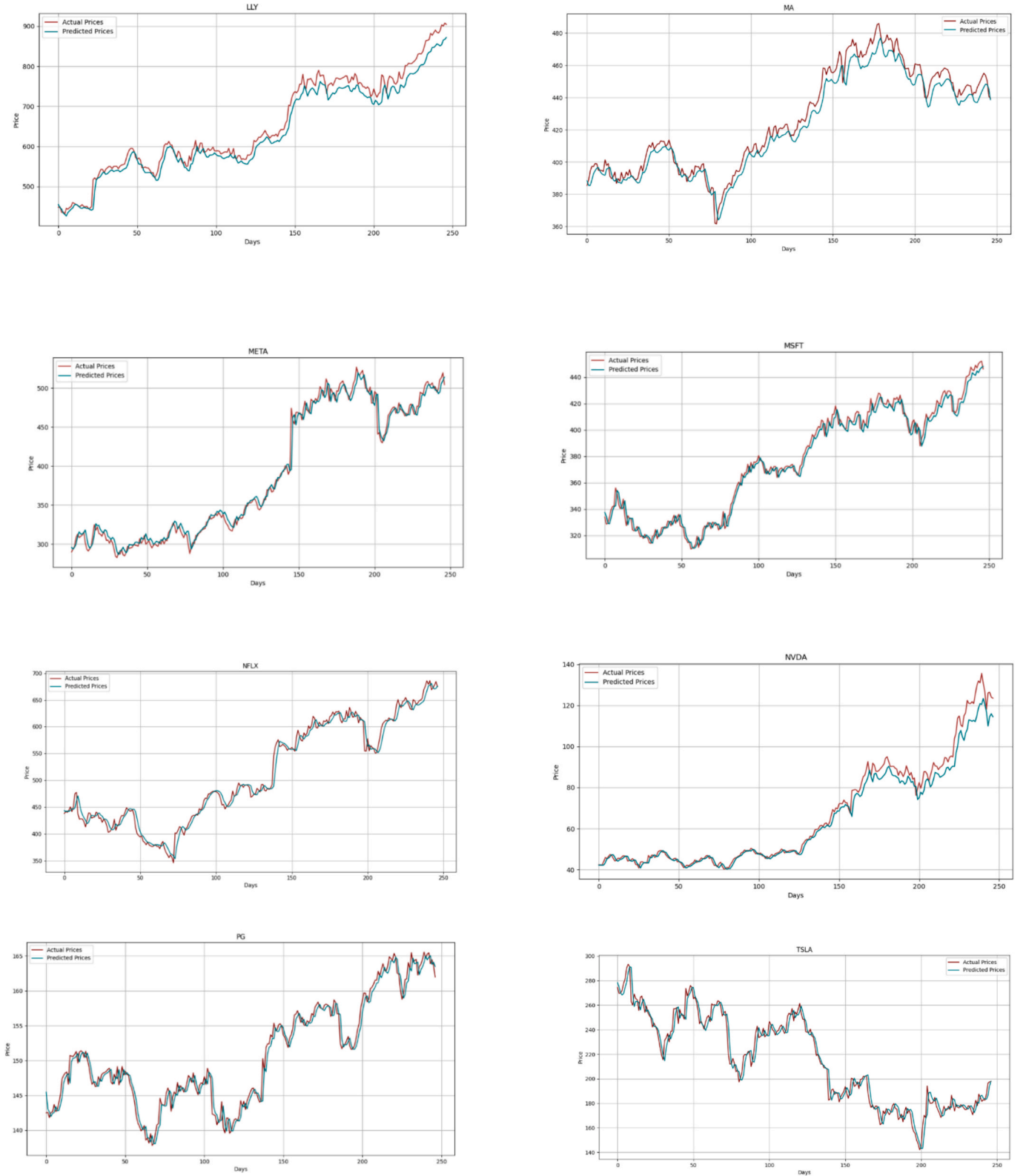


Fig. 1. (continued).

3.3.2. Exploration Stage (Searching the prey)

The golden Jackals know how to detect and pursue the prey. However, if they judge that the prey cannot be caught easily, they may choose to wait and search for another prey. During each epoch (t), the leader of hunting is the male jackal (Eq. (14)), and the female follows

(Eq. (15))

$$Y_1(t) = Y_M(t) - E \bullet |Y_M(t) - rl \bullet Prey(t)| \quad (14)$$

$$Y_2(t) = Y_{FM}(t) - E \bullet |Y_{FM}(t) - rl \bullet Prey(t)| \quad (15)$$

Prey(t) is the position vector of the prey, where the $Y_M(t)$ and $Y_{FM}(t)$

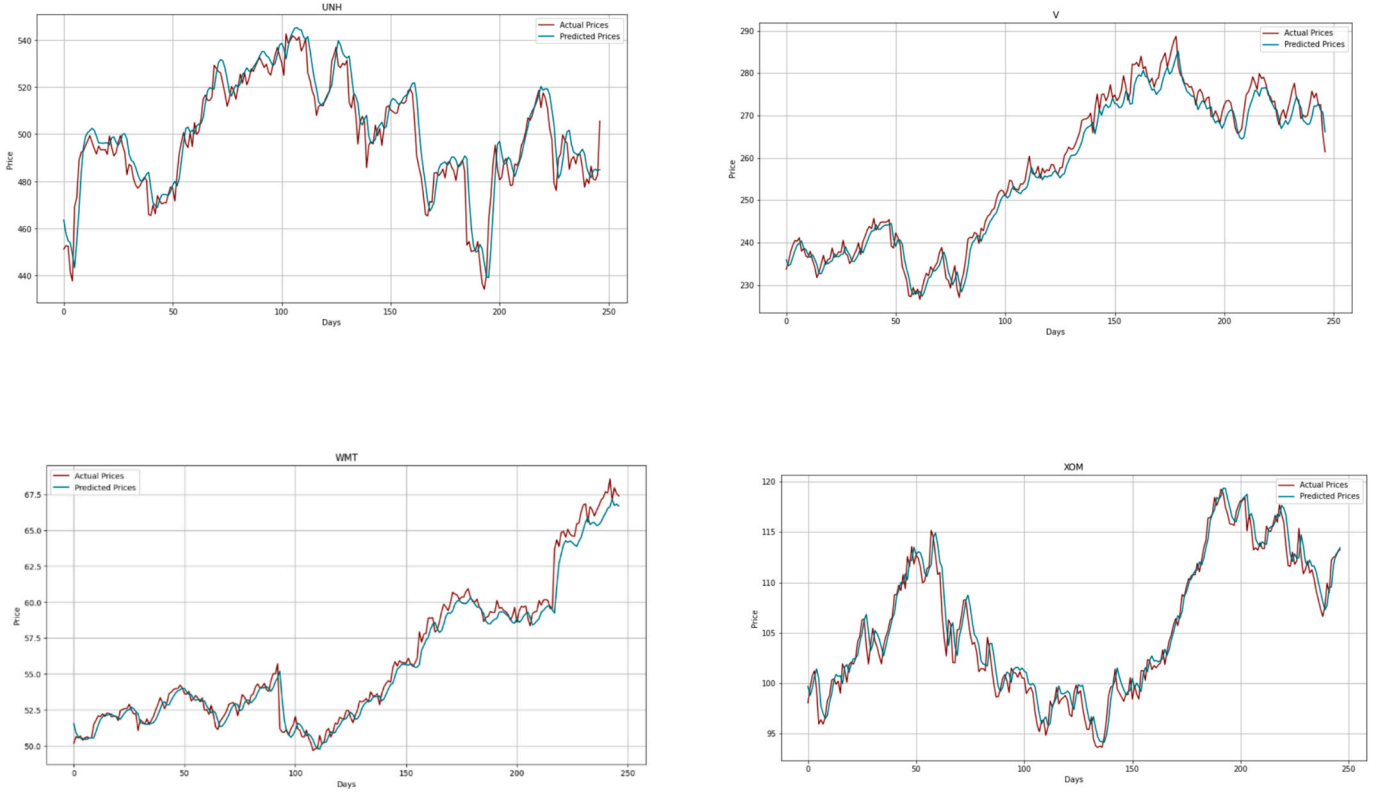


Fig. 1. (continued).

corresponds to the position of male and female jackals respectively. The updated jackal's positions, corresponding to the prey are defined as $Y_1(t)$ and $Y_2(t)$. Prey's evading energy (E) (Eq. (16)) is equal to the multiplication of decreasing energy of the prey (E_0) (Eq. (17)) and the initial state of its energy (E_1) (Eq. (18)). c_1 is a standard fixed parameter equal to 1.5, where T is the total number of epochs. E_1 is linearly decreased from 1.5 to 0 throughout iterations.

$$E = E_1 \bullet E_0 \quad (16)$$

$$E_0 = 2 \bullet r - 1 \quad (17)$$

$$E_1 = c_1 \bullet (1 - 1(t/T)) \quad (18)$$

The variable "rl" (Eq.19) is a vector of random numbers which are generated based on the Lévy distribution, where LF corresponds to the levy flight function (Eq. (20)). Finally, Eq. (21) defines the updated position of the couple of jackals

$$rl = 0.05 \bullet LF(y) \quad (19)$$

$$LF(y) = 0.01 \bullet (\mu \bullet \sigma) / \left(\left| \nu \left(\frac{1}{\beta} \right) \right| \right); \sigma = \left(\frac{\Gamma(1 + \beta) \bullet \sin\left(\frac{\pi\beta}{2}\right)}{\Gamma\left(\frac{1+\beta}{2}\right) \bullet \beta \bullet (2^{\frac{\beta-1}{2}})} \right)^{\frac{1}{\beta}} \quad (20)$$

$$\mu, \nu \in (0, 1)$$

$$\beta = 1.5$$

Eq. (21), defines the updated position of the couple of jackals

$$Y(t+1) = \frac{Y_1(t) + Y_2(t)}{2} \quad (21)$$

3.3.3. Exploitation stage

As the prey is pursued, its evading energy diminishes, allowing the jackals to encircle it. Finally, the couple of jackals pounce on prey. (Eqs.22 & 23)

$$Y_1(t) = Y_M(t) - E \bullet |rl \bullet Y_M(t) - Prey(t)| \quad (22)$$

$$Y_2(t) = Y_{FM}(t) - E \bullet |rl \bullet Y_{FM}(t) - Prey(t)| \quad (23)$$

The role of "rl" is to promote random behaviour during the exploitation phase, encouraging exploration and escape of local optima. This factor contributes to bypassing the sluggishness of local optima, especially during the final iterations. This aspect can be considered a result of obstacles to approaching the prey. Typically, obstacles in the environment hinder jackals' efficient and quick approach to their prey.

3.3.4. Balancing exploration and exploitation

As mentioned earlier in the description of the exploration phase, the jackals may let the prey flee if the judge that it cannot caught easily. This phenomenon allows GJO algorithm to switch from exploration to exploitation. As it is expected during the hunt the prey may be tired as described by Eq. (16). While E_0 decreases from zero to minus one, the prey is experiencing fatigue, whereas when E_0 increases from zero to one, suggests an increase in prey strength. Given, the aforementioned when $|E| > 1$ the jackals will, abort the hunting of the respective prey. On the contrary, when $|E| < 1$ the jackals will pounce through the prey, performing the exploitation phase. Below, we present the pseudocode as provided by the authors (Algorithm 1):

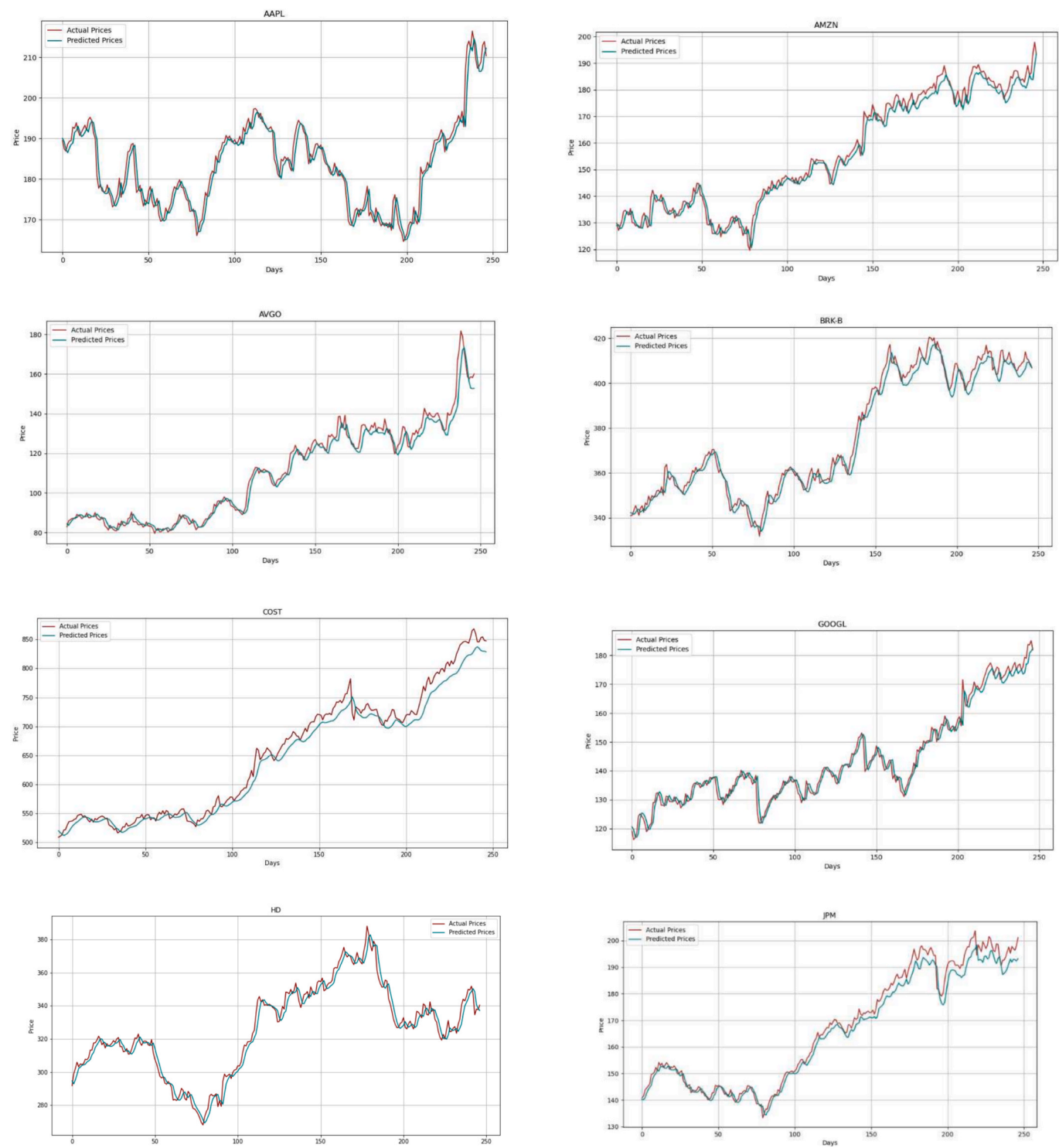


Fig. 2. PSO-GRU|LSTM Results.



Fig. 2. (continued).

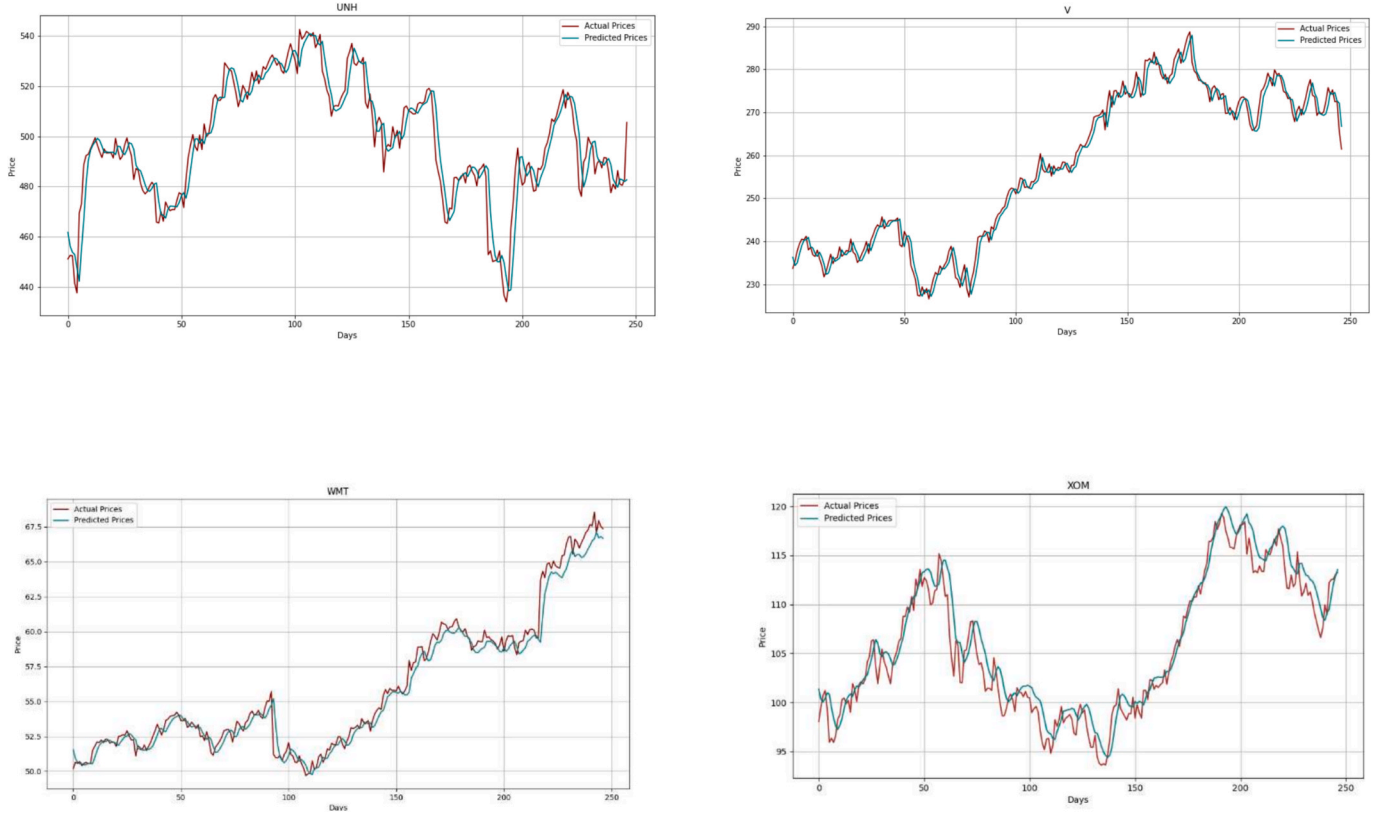


Fig. 2. (continued).

Algorithm 1. (Pseudocode of GJO)**Inputs :** The population size N and maximum number of iterations T **Outputs :** The location of prey and its fitness valueInitialize the random prey population $Y_i (i = 1, 2, \dots, N)$ **While** ($t < T$)

Calculate the fitness values of preys

 Y_1 = best prey (Male Jackal Position) Y_2 = second best prey (Female Jackal Position) **for** (each prey)

Update the evading energy "E" using Eq.(16), Eq.(17) and Eq.(18)

Update "rl" using Eq.(9) and Eq.(10)

if ($|E| \geq 1$) (Explorationphase)

Update the prey position using Eq.(14), Eq.(15) and Eq.(21)

if ($|E| < 1$) (Exploitationphase)

Update the prey position using Eq.(22), Eq.(23) and Eq.(21)

end for $t = t + 1$ **end while** **return** Y_1 **4. Numerical experiments**

In this section, we present the evaluation criteria which are being used in order to evaluate a time series model. Next, we present the dataset as well as the experimental setup. Finally, we present the results, where we discuss the performance of the evaluated models.

4.1. Evaluation criteria

Mean Squared Error (MSE) (Eq. (24)) is defined as the mean of the squared differences between the predicted and actual values. MSE penalizes large deviations between the actual and predicted values due to the squaring of the difference, make it effectively when large deviations exist among the actual and the predicted values. Moreover, as a function

is differentiable, which make it useful for gradient-based optimization methods, such as neural networks models

$$MSE = \frac{1}{N} \left(\sum_i^N (y_i - \hat{y}_i)^2 \right) \quad (24)$$

Although the values of MSE can be quite informative about our model's performance, the root squared MSE (RMSE) can increase interpretability of the models. The RMSE can be seen as the actual value of the difference between the stock price and the predicted values. Given that, we can have a more intuitive performance of our model's performance.

Mean Absolute Error (MAE) (Eq. (25)) is another popular metric which shows the average of the absolute differences between predicted and actual values. In contrast to MSE, it's less sensitive to outliers

$$MAE = \frac{1}{N} \left(\sum_i^N |y_i - \hat{y}_i| \right) \quad (25)$$

Mean Absolute Percentage Error (MAPE) (Eq. (26)) is another well-known evaluation metric for regression task. It measures the accuracy of the model, expressed as a percentage which can be particularly useful when we evaluate the model's performance, across to time-series that have different magnitude /price ranges, in terms of stocks prices. It's worth noting that this specific metric can be quite misleading, when the actual values are close to zero, such as stock prices with small close value, or stock returns. However, in this study we didn't use such stocks, hence it can still be considered as a robust metric.

$$MAPE = \frac{100}{N} \sum_i^N \frac{y_i - \hat{y}_i}{y_i} \quad (26)$$

4.2. Experimental setup

The dataset includes the twenty S&P 500 stocks with the highest

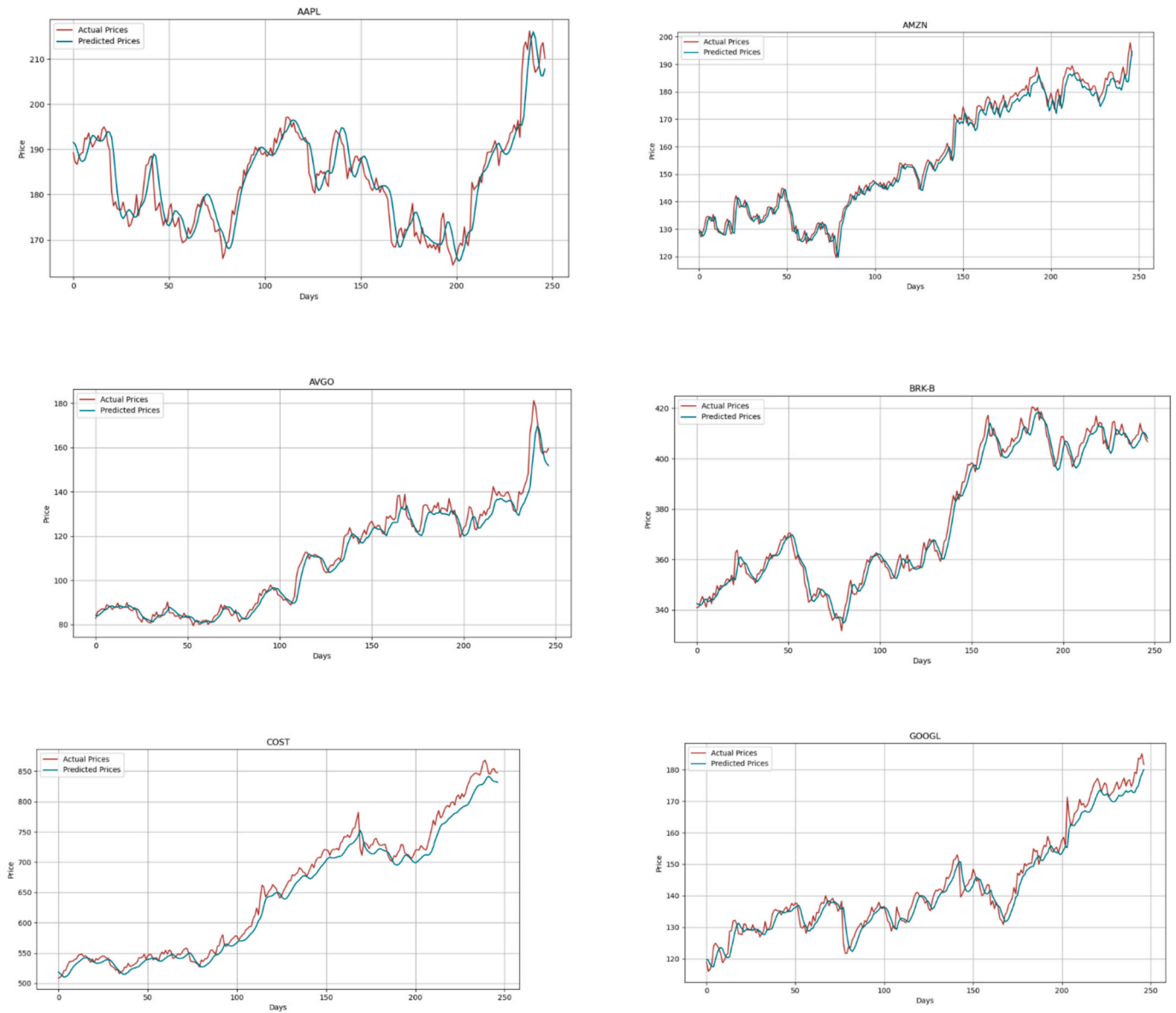


Fig. 3. GA-GRU/LSTM Results.

market capitalization (Table 1). The data were obtained via Yahoo Finance, and the corresponding descriptive statistics are provided in Table 2. Following Gülmez (2023), we utilized a five-year dataset, spanning dates from 1st July 2019 up to 1st July 2024, and we also selected a lookback value as input equal to 20. Furthermore, we compare the performance of GJO with PSO and GA, since these two algorithms are well-established in the existing literature, and have been widely utilized for various tasks, including hyperparameter optimization. The MEALPY package (Van Thieu & Mirjalili, 2023) facilitates the application of these specific algorithms. We chose a relatively small number of epochs and a population size, both set to 30. In general, the literature tends to use higher values for both parameters in most studies. However, our ultimate goal is to evaluate the performance of our proposed approach under a more competitive and constrained scenario, ensuring that it remains effective even with limited computational resources or a restricted timeframe.

For the data transformation, we employed min-max scaling, which is a widely accepted approach for the task of stock price prediction. We utilized the TensorFlow 2.0 (Abadi et al., 2016) package for the

implementation of LSTM and GRU networks, and Scikit-learn (Pedregosa et al., 2011) for preprocessing the data. In addition, we employ early stopping to mitigate the overfitting effect. The metaheuristic-optimized models can select between one or two layers, which could be LSTM or GRU. We also include Dropout layers (Srivastava et al., 2014) to further dodge the overfitting effect. Finally, another key point in our methodology is batch size optimization, which can play a vital role in the model's performance. Most previous works neglect to consider the optimization of this component. In terms of optimizer selection, we use as candidate optimizers, the Root Mean Square Propagation (RMSProp) (Tieleman & Hinton, 2012), the Adaptive Moment Estimation (Adam) (Kingma & Ba, 2014), and the relatively new Rectified Adam (RAdam) (Liu et al., 2019), which has recently demonstrated promising performance (Gkonis & Tsakalos, 2024). Table 3 summarizes the candidate components that will undergo optimization within their respective optimization ranges. In addition, we employ several benchmark models with initial parameter selections, apart from the metaheuristic optimized models (Table 4).

Table 5 provides the results for the metaheuristic-optimized models,



Fig. 3. (continued).

while Tables 6–8 show the results for the respective benchmark models. The optimization performed on the MSE, however we tend to discuss the results based on the RMSE because, as mentioned in the subsection 4.1., it's easier to interpret financial timeseries. Hence, providing both MSE and RMSE tables could be redundant and would not add any additional

information. Figs. 1–3 display the timeseries of metaheuristic optimized models. We have omitted visualizations for the benchmark models to avoid presenting over four-hundred-line graphs, which could overwhelm the reader and lead to confusion. Furthermore, except for RMSE, we also present the MAE and MAPE metrics.

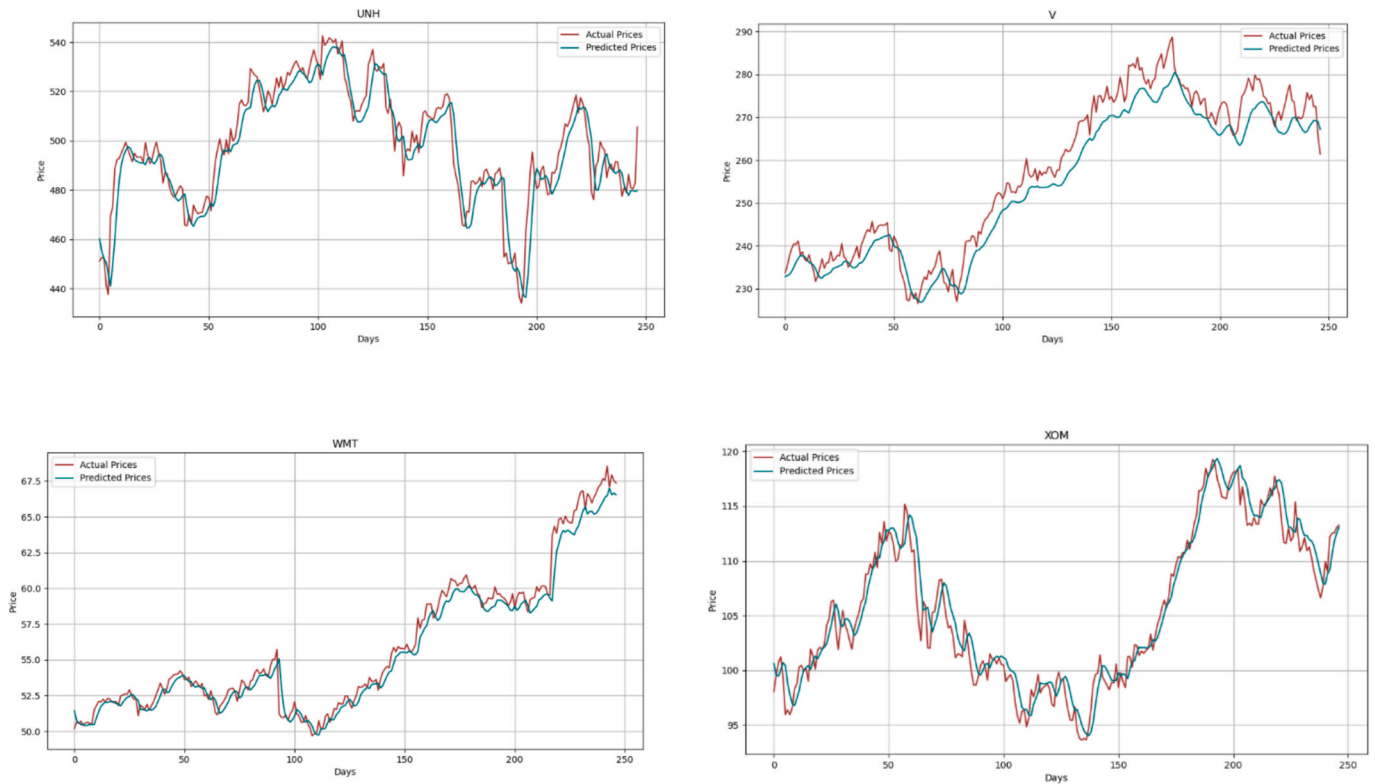


Fig. 3. (continued).

4.3. Results

The results presented in Table 5 demonstrate that metaheuristic-optimized algorithms significantly enhance forecasting accuracy. In terms of the RMSE score, when comparing GJO-GRU | LSTM, PSO-GRU | LSTM, and GA-GRU | LSTM, we observe that GJO-GRU | LSTM outperforms the other models in most cases. Specifically, out of the twenty predicted stocks, GJO-GRU | LSTM delivered the best results in thirteen cases, showcasing its effectiveness. Meanwhile, PSO-GRU | LSTM outperformed the other methods in four cases, while GA-GRU | LSTM provided the optimal results in three cases. Notably, GA-GRU | LSTM achieved superior performance in predicting the prices of LLY and JPM, where it significantly outperformed the other two models. Regarding NVDA, GA-GRU | LSTM achieved the best results, while GJO-GRU | LSTM underperformed. However, in most cases, GA-GRU | LSTM underperformed, particularly in the case of V, where it delivered the weakest results. The PSO-GRU | LSTM model performed well for stocks of GOOGL, NFLX, UNH and V. Regarding the price prediction of MA, PSO-GRU | LSTM delivered the worst performance compared to the other two models.

However, in the thirteen cases where GJO-GRU | LSTM was the best model, it achieved superior accuracy, particularly for COST, MSFT, META, and XOM. It is also worth noting that for HD price prediction, although GJO-GRU | LSTM achieved the highest prediction accuracy, all metaheuristic-optimized models performed well. Additionally, in some cases, GJO-GRU | LSTM and PSO-GRU | LSTM exhibited similar performance, such as in AAPL and TSLA, while GA-GRU | LSTM performed similarly to GJO-GRU | LSTM in forecasting MA stock price, where PSO-GRU | LSTM underperformed. We observed that even the relatively small number of epochs and population size, we still achieved promising results. However, we acknowledge that increasing these two components could further enhance forecasting quality. This is particularly evident in cases like COST, where the best predictability among benchmark models was achieved by GRU-RMSProp, with an RMSE of 33.421—which is significantly inferior compared to metaheuristic-

optimized models, especially the proposed GJO-GRU | LSTM.

Regarding the benchmark models, we observe that they do not follow a standard pattern. Someone might argue that we could achieve good performance using a default architecture. However, we want to prove a point. There is no optimal architecture for predicting a stock price. Various researchers try to deploy optimal architectures for this task. Given the performance of the benchmark models, it's clear that it is impossible to reach a clear decision. For example, among the benchmark models, we acquired the highest predictability performance for LLY and MSFT with GRU-RAdam. Conversely, this particular architecture underperformed when we attempted to predict the stock prices of GOOGL and AVGO. Simultaneously, regarding AAPL and AMZN, we observe a shift in the performance of the two most effective models. In one scenario, the LSTM-RMSProp ranks first, followed by the GRU-RMSProp, while the other scenario reverses the results.

We could probably state that no stacked architecture, can provide good results, which indeed happens in most cases. However, with hybrid stacked LSTM-GRU or GRU-LSTM, there were cases that achieved promising performance, such as BRK-B, where LSTM-GRU-RAdam yielded the highest prediction accuracy. Although we set a standard number of units and batch size in our benchmark models, we can conclude that there is no dominant architecture, since each time series has its own characteristics. Regarding optimizers, although RAdam was employed in scenarios for different architectures providing the best performance, including also LSTM-RAdam in the case of JPM, which surpassed both metaheuristic-optimized deep learning models, at the same time was also the optimizer in the cases with the worst performance in half cases.

Somebody could question the use of metaheuristic optimization techniques and their importance in this regard, as well as the importance of this research, including other similar works. Considering the optimization ranges of Table 3., we observe that the combination of available components leads to over five million different neural network architectures. Given that the evaluated data comprises twenty stocks, this leads to over one hundred million different deep learning models.

However, as previously discussed, the benchmark model's results, indicate the absence of a dominant architecture. It is evident that the use of optimizing techniques, particularly metaheuristics, is essential for achieving optimal forecasting models, eliminating the need of manual parameter selection by researchers or practitioners. Optimizing the architecture, including important components such as optimizers, number of units, layers, and batch size, can be critical for obtaining a robust deep learning model for accurate stock prediction. The proposed GJO-GRU | LSTM seems like a promising approach for that purpose since it can perform better in most cases.

5. Conclusion

In this work, we proposed the novelty GJO-GRU | LSTM, which consists of an optimized deep learning architecture for the task of accurate forecasting of stock prices via Golden Jackal Optimizer. We compared the proposed model's performance with two optimized architectures utilizing PSO and GA. The results demonstrated that GJO is able to provide better performance in contrast to GJO. Moreover, we compared the performance of metaheuristic-optimized deep learning models with eighteen other benchmark models with default hyperparameters. We evaluated the proposed models based on a five-year dataset comprising S&P500's stocks with the largest capitalization, evaluating primarily their performance based on RMSE. The results revealed that not only is hyperparameter optimization important, but it's also vital to select the optimal architecture for time series. According to benchmark models, there is no dominant architecture or optimizer that can provide optimal results for all cases. This finding shows that instead of optimizing LSTM or GRU, which includes one or two layers, it is beneficial to include both (GRU | LSTM) architectures into consideration, except for unit selection, dropout layer, and optimizer. Another contribution of this study is the optimization of the batch size, which, despite being a crucial component for neural network models, has not received much attention in the existing literature. This work contributes to the field of stock price prediction. Moreover, it's the first time that GJO is being proposed in this regard. The proposed approach contributes significantly to the existing literature in the fields of finance, metaheuristic optimization, and machine learning.

However, this work does not come without limitations. As mentioned in subsection 4.2., we used a relatively small number of epochs and population size, both of which were equal to thirty. The authors did not select this simple setup without understanding its limitations. Using a larger number of these components could be critical to the performance of metaheuristic optimization algorithms. However, it could also result in an increase in computational time and resources. Future directions could include using a larger number of these two components. Moreover, we chose to compare the performance of GJO with PSO and GA, two widely evaluated metaheuristic swarm-intelligence algorithms in the literature. Comparing the performance with more metaheuristic algorithms could further improve our findings' robustness. Moreover, several studies in the literature propose improved modifications of GJO, which could be evaluated. Additionally, exploring new modifications of GJO by integrating a fuzzy self-tuning approach, similar to that used in the Flying Fox Optimizer (Zervoudakis & Tsafarakis, 2023), could be a promising direction for future research. Finally, a larger sample size that includes more stocks and compares with more benchmark models would further enhance the validity of our findings.

Funding

The authors received no specific funding for this work.

Code availability

Not applicable.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data was acquired via Yahoo Finance (<https://finance.yahoo.com/>).

References

- Abadi, M., A. Agarwal, P. Barham, et al. 2016. TensorFlow: LargeScale Machine Learning on Heterogeneous Distributed Systems. arXiv Preprint arXiv:1603.04467.
- Abd Elaziz, M., Dahou, A., Abualigah, L., Yu, L., Alshinwan, M., Khasawneh, A. M., & Lu, S. (2021). Advanced metaheuristic optimization techniques in applications of deep neural networks: A review. *Neural Computing and Applications*, 1–21. <https://doi.org/10.1007/s00521-021-05960-5>
- Abdollahzadeh, B., Khodadadi, N., Barshandeh, S., Trojovský, P., Gharehchopogh, F. S., El-kenawy, E. S. M., & Mirjalili, S. (2024). Puma optimizer (PO): A novel metaheuristic optimization algorithm and its application in machine learning. *Cluster Computing*, 27, 5235–5283. <https://doi.org/10.1007/s10586-023-04221-5>
- Adebiyi, A. A., Adewumi, A. O., & Ayo, C. K. (2014). Comparison of ARIMA and artificial neural networks models for stock price prediction. *Journal of Applied Mathematics*, 2014(1), Article 614342. <https://doi.org/10.1155/2014/614342>
- Akay, B., Karaboga, D., & Akay, R. (2022). A comprehensive survey on optimizing deep learning models by metaheuristics. *Artificial Intelligence Review*, 55(2), 829–894. <https://doi.org/10.1007/s10462-021-09992-0>
- Almaafi, A., Bajaba, S., & Alnori, F. (2023). Stock price prediction using ARIMA versus XGBoost models: The case of the largest telecommunication company in the Middle East. *International Journal of Information Technology*, 15(4), 1813–1818. <https://doi.org/10.1007/s41870-023-01260-4>
- Andriopoulos, D., Doumpos, M., Pardalos, P. M., & Zopounidis, C. (2019). Computational approaches and data analytics in financial services. *Journal of the Operational Research Society*, 70(10), 1579–1580. <https://doi.org/10.1080/01605682.2019.1649932>
- Awartani, B. M., & Corradi, V. (2005). Predicting the volatility of the S&P-500 stock index via GARCH models: The role of asymmetries. *International Journal of forecasting*, 21(1), 167–183. <https://doi.org/10.1016/j.ijforecast.2004.08.003>
- Ballings, M., Van den Poel, D., Hespeels, N., & Gryp, R. (2015). Evaluating multiple classifiers for stock price direction prediction. *Expert systems with Applications*, 42 (20), 7046–7056. <https://doi.org/10.1016/j.eswa.2015.05.013>
- Bosworth, B., Hymans, S., & Modigliani, F. (1975). The stock market and the economy. *Brookings Papers on Economic Activity*, 1975(2), 257–300. <https://doi.org/10.2307/2534104>
- Bischi, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., & Lindauer, M. (2023). Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 13 (2), e1484.
- Chiang, T. C., & Doong, S. C. (2001). Empirical analysis of stock returns and volatility: Evidence from seven Asian stock markets based on TAR-GARCH model. *Review of Quantitative Finance and Accounting*, 17, 301–318. <https://doi.org/10.1023/A:1012296727217>
- Chong, C. W., Ahmad, M. I., & Abdullah, M. Y. (1999). Performance of GARCH models in forecasting stock market volatility. *Journal of forecasting*, 18(5), 333–343. [https://doi.org/10.1002/\(SICI\)1099-131X\(199909\)18:5<333::AID-FOR742>3.0.CO;2-K](https://doi.org/10.1002/(SICI)1099-131X(199909)18:5<333::AID-FOR742>3.0.CO;2-K)
- Cho, K., Van Merriënboer, B., Gulcehre, Ç., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078.
- Chou, J. S., Nguyen, N. M., & Chang, C. P. (2022). Intelligent candlestick forecast system for financial time-series analysis using metaheuristics-optimized multi-output machine learning. *Applied Soft Computing*, 130, Article 109642. <https://doi.org/10.1016/j.asoc.2022.109642>
- Chopra, N., & Ansari, M. M. (2022). Golden jackal optimization: A novel nature-inspired optimizer for engineering applications. *Expert Systems with Applications*, 198, Article 116924. <https://doi.org/10.1016/j.eswa.2022.116924>
- Chung, H., & Shin, K. S. (2018). Genetic algorithm-optimized long short-term memory network for stock market prediction. *Sustainability*, 10(10), 3765. <https://doi.org/10.3390/su10103765>
- Di Persio, L., & Honchar, O. (2017). Recurrent neural networks approach to the financial forecast of Google assets. *International journal of Mathematics and Computers in simulation*, 11, 7–13.
- Dutta, A., Kumar, S., & Basu, M. (2020). A gated recurrent unit approach to bitcoin price prediction. *Journal of risk and financial management*, 13(2), 23. <https://doi.org/10.3390/jrfm13020023>
- Gao, Y., Wang, R., & Zhou, E. (2021). Stock prediction based on optimized LSTM and GRU models. *Scientific Programming*, 2021(1), Article 4055281. <https://doi.org/10.1155/2021/4055281>
- Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to forget: Continual prediction with LSTM. *Neural computation*, 12(10), 2451–2471. <https://doi.org/10.1162/089976600300015015>

- Gülmez, B. (2023). Stock price prediction with optimized deep LSTM network with artificial rabbits optimization algorithm. *Expert Systems with Applications*, 227, Article 120346. <https://doi.org/10.1016/j.eswa.2023.120346>
- Ertelice, O., & Kalayci, C. B. (2018). A survey of swarm intelligence for portfolio optimization: Algorithms and applications. *Swarm and evolutionary computation*, 39, 36–52. <https://doi.org/10.1016/j.swevo.2018.01.009>
- Fang, W., Zhang, S., & Xu, C. (2024). Improving prediction efficiency of Chinese stock index futures intraday price by VIX-Lasso-GRU Model. *Expert Systems with Applications*, 238, Article 121968. <https://doi.org/10.1016/j.eswa.2023.121968>
- Franses, P. H., & Van Dijk, D. (1996). Forecasting stock market volatility using (non-linear) Garch models. *Journal of forecasting*, 15(3), 229–235. [https://doi.org/10.1002/\(SICI\)1099-131X\(199604\)15:3<229::AID-FOR620>3.0.CO;2-3](https://doi.org/10.1002/(SICI)1099-131X(199604)15:3<229::AID-FOR620>3.0.CO;2-3)
- Fama, E. F. (1995). Random walks in stock market prices. *Financial analysts journal*, 51(1), 75–80. <https://doi.org/10.2469/faj.v51.n1.1861>
- Fan, L. (2022). Mobile investment technology adoption among investors. *International Journal of Bank Marketing*, 40(1), 50–67. <https://doi.org/10.1108/IJBM-11-2020-0551>
- Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European journal of operational research*, 270(2), 654–669. <https://doi.org/10.1016/j.ejor.2017.11.054>
- Gkonis, V., & Tsakalos, I. (2024). Deep Dive Into Churn Prediction in the Banking Sector: The Challenge of Hyperparameter Selection and Imbalanced Learning. *Journal of Forecasting*, 44(2), 281–296. <https://doi.org/10.1002/for.3194>
- Göçken, M., Özçalıcı, M., Boru, A., & Dosdoğru, A. T. (2016). Integrating metaheuristics and artificial neural networks for improved stock price prediction. *Expert Systems with Applications*, 44, 320–331. <https://doi.org/10.1016/j.eswa.2015.09.029>
- Godfrey, M. D., Granger, C. W., & Morgenstern, O. (1964). The Random-Walk Hypothesis of Stock Market Behavior a. *Kyklos*, 17(1), 1–30. <https://doi.org/10.1111/j.1467-6435.1964.tb02458.x>
- Guresen, E., Kayakutlu, G., & Daim, T. U. (2011). Using artificial neural network models in stock market index prediction. *Expert systems with Applications*, 38(8), 10389–10397. <https://doi.org/10.1016/j.eswa.2011.02.068>
- Hamayel, M. J., & Owda, A. Y. (2021). A novel cryptocurrency price prediction model using GRU, LSTM and bi-LSTM machine learning algorithms. *AI*, 2(4), 477–496. <https://doi.org/10.3390/ai2040030>
- Hochreiter, S. (1991). Untersuchungen zu dynamischen neuronalen Netzen. *Diploma, Technische Universität München*, 91(1), 31.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Holland, J. H. (1992). *Genetic algorithms. Scientific american*, 267(1), 66–73.
- Jarrett, J. E., & Kyper, E. (2011). ARIMA modeling with intervention to forecast and analyze Chinese stock prices. *International Journal of Engineering Business Management*, 3, 17. <https://doi.org/10.5772/50938>
- Ji, Y., Liew, A. W. C., & Yang, L. (2021). A novel improved particle swarm optimization with long-short term memory hybrid model for stock indices forecast. *IEEE Access*, 9, 23660–23671. <https://doi.org/10.1109/ACCESS.2021.3056713>
- Jiang, W. (2021). Applications of deep learning in stock market prediction: Recent progress. *Expert Systems with Applications*, 184, Article 115537. <https://doi.org/10.1016/j.eswa.2021.115537>
- Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv Preprint arXiv:1412.6980.
- Kennedy, J., & Eberhart, R. (1995). November). *Particle swarm optimization. In Proceedings of ICNN'95-international conference on neural networks (Vol. 4., 1942–1948.*
- Kumar, G., Singh, U. P., & Jain, S. (2022a). An adaptive particle swarm optimization-based hybrid long short-term memory model for stock price time series forecasting. *Soft Computing*, 26(22), 12115–12135. <https://doi.org/10.1007/s00500-022-07451-8>
- Kumar, G., Singh, U. P., & Jain, S. (2022b). Swarm intelligence based hybrid neural network approach for stock price forecasting. *Computational Economics*, 60(3), 991–1039. <https://doi.org/10.1007/s10614-021-10176-9>
- Kumbure, M. M., Lohrmann, C., Luukka, P., & Porras, J. (2022). Machine learning techniques and data for stock market forecasting: A literature review. *Expert Systems with Applications*, 197, Article 116659. <https://doi.org/10.1016/j.eswa.2022.116659>
- Liu, L., H. Jiang, P. He. et al. (2019). On the Variance of the Adaptive Learning Rate and Beyond. arXiv Preprint arXiv:1908.03265.
- Liu, K., Zhou, J., & Dong, D. (2021). Improving stock price prediction using the long short-term memory model combined with online social networks. *Journal of Behavioral and Experimental Finance*, 30, Article 100507. <https://doi.org/10.1016/j.jbef.2021.100507>
- Liu, J., & Wang, Z. (2024). Financial distress prediction using an improved particle swarm optimization wrapper feature selection method and tree boosting ensemble. *Journal of the Operational Research Society*, 1–24. <https://doi.org/10.1080/01605682.2024.2385467>
- Liu, X., Wu, Y., Luo, M., & Chen, Z. (2024). Stock price prediction for new energy vehicle companies based on multi-source data and hybrid attention structure. *Expert Systems with Applications*, 255, Article 124787. <https://doi.org/10.1016/j.eswa.2024.124787>
- Lv, D., Yuan, S., Li, M., & Xiang, Y. (2019). An empirical study of machine learning algorithms for stock daily trading strategy. *Mathematical problems in engineering*, 2019(1), Article 7816154. <https://doi.org/10.1155/2019/7816154>
- Makridakis, S., Spiliotis, E., Assimakopoulos, V., Semenoglou, A. A., Mulder, G., & Nikolopoulos, K. (2023). Statistical, machine learning and deep learning forecasting methods: Comparisons and ways forward. *Journal of the Operational Research Society*, 74(3), 840–859. <https://doi.org/10.1080/01605682.2022.2118629>
- Marqués, A. I., García, V., & Sánchez, J. S. (2013). A literature review on the application of evolutionary computing to credit scoring. *Journal of the Operational Research Society*, 64(9), 1384–1399. <https://doi.org/10.1057/jors.2012.145>
- Mestel, R., & Gurgul, H. (2003). ARIMA Modeling of Event Induced Stock Price Reactions in Austria. *Central European Journal of Operations Research*, 11(4).
- Morales-Hernández, A., Van Nieuwenhuysse, I., & Rojas Gonzalez, S. (2023). A survey on multi-objective hyperparameter optimization algorithms for machine learning. *Artificial Intelligence Review*, 56(8), 8043–8093. <https://doi.org/10.1007/s10462-022-10359-2>
- Mu, G., Gao, N., Wang, Y., & Dai, L. (2023). A stock price prediction model based on investor sentiment and optimized deep learning. *IEEE Access*, 11, 51353–51367. <https://doi.org/10.1109/ACCESS.2023.3278790>
- Nabipour, M., Nayeri, P., Jabani, H., Mosavi, A., & Salwana, E. (2020). Deep learning for stock market prediction. *Entropy*, 22(8), 840. <https://doi.org/10.3390/e22080840>
- Najand, M. (2002). Forecasting stock index futures price volatility: Linear vs. nonlinear models. *Financial Review*, 37(1), 93–104. <https://doi.org/10.1111/1540-6288.00006>
- Narayanan, R., & Ganesh, N. (2024). A Comprehensive Review of Metaheuristics for Hyperparameter Optimization in Machine Learning. *Metaheuristics for Machine Learning: Algorithms and Applications*, 37–724. <https://doi.org/10.1002/9781394233953.ch2>
- Olah, C. (2015). Understanding LSTM Networks. <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. the Journal of machine learning research, 12, 2825–2830.
- Rajwar, K., Deep, K., & Das, S. (2023). An exhaustive review of the metaheuristic algorithms for search and optimization: Taxonomy, applications, and open challenges. *Artificial Intelligence Review*, 56(11), 13187–13257. <https://doi.org/10.1007/s10462-023-10470-y>
- Ribeiro, G. T., Santos, A. A. P., Mariani, V. C., & dos Santos Coelho, L. (2021). Novel hybrid model based on echo state neural network applied to the prediction of stock price return volatility. *Expert Systems with Applications*, 184, Article 115490. <https://doi.org/10.1016/j.eswa.2021.115490>
- Shahi, T. B., Shrestha, A., Neupane, A., & Guo, W. (2020). Stock price forecasting with deep learning: A comparative study. *Mathematics*, 8(9), 1441. <https://doi.org/10.3390/math8091441>
- Shahvaroughi Farahani, M., & Razavi Hajiagha, S. H. (2021). Forecasting stock price using integrated artificial neural network and metaheuristic algorithms compared to time series models. *Soft computing*, 25(13), 8483–8513. <https://doi.org/10.1007/s00500-021-05775-5>
- Sirisha, U. M., Belavagi, M. C., & Attigeri, G. (2022). Profit prediction using ARIMA, SARIMA and LSTM models in time series forecasting: A comparison. *IEEE Access*, 10, 124715–124727. <https://doi.org/10.1109/ACCESS.2022.3224938>
- Siami-Namini, S., & Namin, A. S. (2018). Forecasting economics and financial time series: ARIMA vs. LSTM. arXiv preprint arXiv:1803.06386.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1), 1929–1958.
- Stevenson, S. (2007). A comparison of the forecasting ability of ARIMA models. *Journal of Property Investment & Finance*, 25(3), 223–240. <https://doi.org/10.1108/14635780710746902>
- Tieleman, T., & Hinton, G. (2012). “Lecture 6.5-rmsprop: Divide the Gradient by a Running Average of Its Recent Magnitude.” COURSE: Neural Networks for. *Machine Learning*, 4(2), 26–30.
- Van Thieu, N., & Mirjalili, S. (2023). MEALPY: An open-source library for latest metaheuristic algorithms in Python. *Journal of Systems Architecture*, 139, Article 102871. <https://doi.org/10.1016/j.sysarc.2023.102871>
- Wu, S., Liu, Y., Zou, Z., & Weng, T. H. (2022). S_I LSTM: Stock price prediction based on multiple data sources and sentiment analysis. *Connection Science*, 34(1), 44–62. <https://doi.org/10.1080/09540091.2021.1940101>
- Xiaopeng, Y. A. N. G. (2024). Prediction of Financial Markets Utilizing an Innovatively Optimized Hybrid Model: A Case Study of the Hang Seng Index. *International Journal of Advanced Computer Science & Applications*, 15(4). <https://doi.org/10.14569/IJACSA.2024.0150415>
- Yang, L., & Shami, A. (2020). On hyperparameter optimization of machine learning algorithms: Theory and practice. *Neurocomputing*, 415, 295–316. <https://doi.org/10.1016/j.neucom.2020.07.061>
- Yu, P., & Yan, X. (2020). Stock price prediction based on deep neural networks. *Neural Computing and Applications*, 32(6), 1609–1628. <https://doi.org/10.1007/s00521-019-04212-x>
- Yu, Y., Lin, Y., Hou, X., & Zhang, X. (2023). Novel optimization approach for realized volatility forecast of stock price index based on deep reinforcement learning model. *Expert Systems with Applications*, 233, 120880. doi: 10.1016/j.eswa.2023.120880.
- Zervoudakis, K., & Tsafarakis, S. (2020). A mayfly optimization algorithm. *Computers & Industrial Engineering*, 145, Article 106559. <https://doi.org/10.1016/j.cie.2020.106559>
- Zervoudakis, K., & Tsafarakis, S. (2023). A global optimizer inspired from the survival strategies of flying foxes. *Engineering with Computers*, 39, 1583–1616. <https://doi.org/10.1007/s00366-021-01554-w>